Class-Incremental Learning for Semantic Segmentation in Aerial Imagery via Distillation in All Aspects

Lianlei Shan, *Graduate Student Member, IEEE*, Weiqiang Wang, *Member, IEEE*, Ke Lv, *Member, IEEE*, and Bin Luo, *Senior Member, IEEE*

***Abstract*— Incremental learning using neural networks achieves great success in semantic segmentation but still suffers from catastrophic forgetting. In this article, we propose an effective class-incremental segmentation method without storing old data. To alleviate the issue of forgetting, we present two important modules, i.e., the deep feature distillation (DFD) module and the label mixed (LM) module. The DFD module is established to learn a good feature representation of old classes by distilling a new compact feature representation from different layers of networks. The proposed LM module first identifies the examples (pixels) of old classes with high confidences utilizing the output of old models, and then, they are combined with examples of new classes to supervise the training of new models, which can achieve a good balance between learning new classing and avoiding forgetting old ones. Our ablation studies show that the DFD module and the LM module can make the learning network obtain 6.2% and 15% performance gains [mean Intersection over Union (mIOU)], respectively. Furthermore, by introducing the supervision of output distillation loss, we compare our method with several state-of-the-art methods in the extensive experiments, and the experimental results all show that our method is significantly superior to them on the dataset of aerial images.**

***Index Terms*— Class-incremental segmentation, deep feature distillation (DFD) module, incremental learning, label mixed (LM) module, semantic segmentation.**

I. Introduction

THE rapid development of satellite sensors enables us to capture a large number of aerial images. Automatically generating analysis or annotation results for them has been an open hot research issue due to its critical importance in agriculture, navigation, environmental management, urban monitoring, mapping, and so on. Among them, semantic segmentation for aerial images, also called land cover classification, aims to predict a class for each pixel in an image. As the emergence of the convolutional neural networks (CNNs), many semantic segmentation models have been presented to improve the segmentation accuracy, such as U-Net [1], PSPNet [2], and Deeplab series [3], or increase the prediction speed, such as ICNet [4]. Different from natural images, the target size of aerial images varies greatly, and the texture information is also relatively complex, so some segmentation networks specifically for aerial images are presented, such as GLNet [5], [6], DENet [7], UHRSNet [8], and others [9]–[14]. For example, *et al.* [9] propose a unified annotation framework by combining discriminative feature learning and feature transferring. In [10], a dynamic learning strategy is proposed to progressively learn the object detectors. Cheng *et al.* [11] propose a simple but effective method to learn discriminative CNNs.

When we apply the above methods for aerial images, we must completely collect all the classes at one time; otherwise, a model has to be rebuilt from scratch when a new class is added. Since different environments possibly have different geographical classes, it is difficult to collect all classes at one time. Apparently, discarding the obtained network models means the waste of a huge amount of computing resources used to train the old models when new classes are encountered, and sometimes, data for training old models cannot be guaranteed to be obtained. Thus, the incremental learning way in semantic segmentation of aerial images is an appropriate solution for saving computing resources or reducing data requirements. Although incremental learning has to face catastrophic forgetting [15], [16], i.e., when learning a new class, the network usually degrades the performance of classifying old classes, class incremental learning is of great practical significance in ensuring the extensibility of models and the segmentation ability of a large number of classes.

Fig. 1 shows the overall process of class-incremental semantic segmentation. In Fig. 1, a new class (or a few classes) is added and trained at each step. The input of network-1 is the images of the class and the corresponding annotations. Different from the network of the first step, the following networks also need to learn from previously trained networks. In each step, only the annotations and images of the current new class are provided in the current step, and the other classes are treated as the background class. The initial network

Manuscript received July 8, 2021; revised October 11, 2021 and November 23, 2021; accepted November 29, 2021. Date of publication December 13, 2021; date of current version March 1, 2022. This work was supported in part by the NSFC Key Projects of International (Regional) Cooperation and Exchanges under Grant 61860206004 and in part by the NSFC Projects under Grant 61976201. *(Corresponding author: Weiqiang Wang.)*

Lianlei Shan and Weiqiang Wang are with the School of Computer Science and Technology, University of Chinese Academy of Sciences, Beijing 100049, China (e-mail: shanlianlei18@mails.ucas.edu.cn; wqwang@ucas.ac.cn).

Ke Lv is with the School of Engineering Science, University of Chinese Academy of Sciences, Beijing 100049, China.

Bin Luo is with the MOE Key Laboratory of Signal Processing and Intelligent Computing, School of Computer Science and Technology, Anhui University, Hefei 230601, China.

Digital Object Identifier 10.1109/TGRS.2021.3135456

1558-0644 © 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 12:56:56 UTC from IEEE Xplore. Restrictions apply.

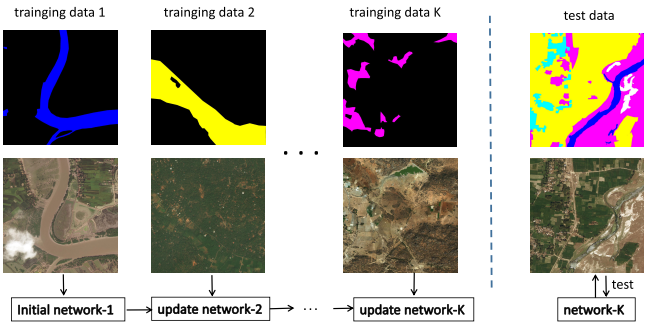


Fig. 1. Whole process of class-incremental segmentation learning. The training phase is on the left of the dotted line, and the testing phase is on the right. K is the total number of classes in the dataset. In each step, there is only the label of the class that is to be learned in the current step.

parameters of each step come from the network in the previous step. During the testing phase, all classes are tested, and the tested model is the network generated by the final step.

The previous incremental learning-based mainstream methods maintain the classification capability of previous classes by knowledge distillation. The distillation only appears in the output space. For example, Li *et al.* [17] propose a twin-auxiliary model that adds an auxiliary binary classification task, where the distillation is performed at the output level. In our work, the distillation is extended to feature levels besides the output level, and we carry out both soft distillation (ordinary distillation) and hard distillation (LM module) for the output level. Our network in class-incremental learning has a completely consistent structure in different training steps to provide a large operating space for distillation. In our network, a multilevel deep feature distillation (DFD) module is designed so that it can maintain the feature representation capacity of previous classes to the maximum extent.

Compared with the classification task at the image level, semantic segmentation predicts the classes at the pixel level. For aerial images, many classes generally coexist in an image. A lot of in-depth experiments show that the reliable predictions of the old model for old classes can provide trustable pseudolabels. Based on this, the label mixed (LM) module is designed to combine the output of the old model after certain filtering with the true label of the new class as the comprehensive supervised information. The LM module can be thought of as a hard distillation if we take the output of old models with high confidence as true labels.

The main contributions of this article can be summarized as follows.

1) We propose a class-incremental segmentation framework for aerial images without storing old training data. A powerful DFD module is established to learn the good feature representation of old classes by distilling a new compact feature representation from different layers of networks.
2) An LM module is presented, which can combine the examples (pixels) of old classes with high confidences with examples of new classes to supervise the training of new models. It can effectively achieve a good balance between learning new classing and avoiding forgetting old ones.
3) Besides feature distillation loss and classification loss, the output distillation [18] is also introduced in our incremental learning process, and the extensive experiments show that all of them make our method significantly superior to some state-of-the-art methods on the dataset of aerial images.

II. Related Work

A. Class-Incremental Semantic Segmentation

Tasar *et al.* [19] present a retraining strategy to select relevant samples from old data and applies it to aerial images. Ozdemir *et al.* [20] and Ozdemir and Goksel [21] propose a similar method to segment medical images. The above methods are all required to store old data, which may cause the risk of data leakage, and sometimes, old data are difficult to obtain practically. Feng *et al.* [22] propose an incremental learning strategy suitable for different datasets, and good results are obtained. Michieli and Zanuttigh [23] present to apply incremental learning in the semantic segmentation of images where soft labels for old classes are directly provided by old classifiers when learning a new class. Comparatively, our method makes some improvements on it, and only the output labels of old classes with high confidence are selected as the training data for a new classifier. Klingner *et al.* [24] propose to combine distillation loss and classification loss in autonomous driving scenarios. Cermelli *et al.* [18] find the problem of class inconsistencies in different training stages and propose an effective class correction method, and the related techniques are also adopted in our system.

B. Incremental Learning

Incremental learning, also known as continuous learning, aims to learn a sequence of tasks and expects that it does not degrade the capability of performing old tasks when learning a new task. For the image classification task [25], the issue of catastrophic forgetting [26] has been extensively studied. Incremental learning methods can be divided into three classes: replay-based methods [27]–[30], parameter isolation-based methods [31], [32], and regularization-based methods [33]–[37]. In the replay-based approaches, examples of previous tasks are stored [27], [29], [38] or generated [28], [30] and then replayed when learning a new task. For synthetic aperture radar (SAR) images, Dang *et al.* [39] present the strategy to select previous key samples, and Dang *et al.* [40] propose an ensemble classifier. The methods based on parameter isolation [31], [32] allocate a subset of parameters for each task to prevent forgetting. Regularization-based methods can be divided into prior-based methods and data-based methods. The former [33]–[35], [41] defines knowledge as parameter values and punishes changes in important parameters of the old tasks when learning a new task. The latter [36], [37] uses distillation [42], i.e., using the distance between the activations generated by old networks and new networks as a regularization term to relieve catastrophic forgetting. Distillation is one

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 12:56:56 UTC from IEEE Xplore. Restrictions apply.

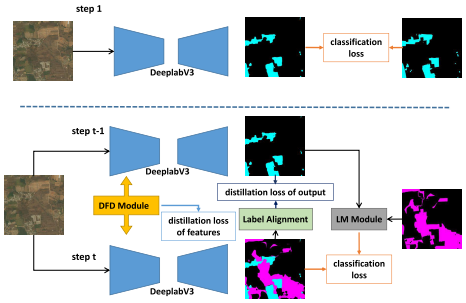


Fig. 2. Overview of the proposed approach. The top half of the dotted line is the first step of the training, and the bottom is the second and subsequent training. The first step (step 1) represents the initial training of classes. The blue parts represent the encoder and the decoder of the segmentation network. The training of the first step is the same as that of the ordinary segmented network, except that the label contains only the class to be learned. In step t, the difference with the first step is that network needs to learn the ability of step $t-1$ model through the DFD module and obtain the comprehensive label through the LM module. The network structures in different steps keep the same. There are three losses in the second and subsequent training steps: classification loss, distillation loss of output, and distillation loss of features.

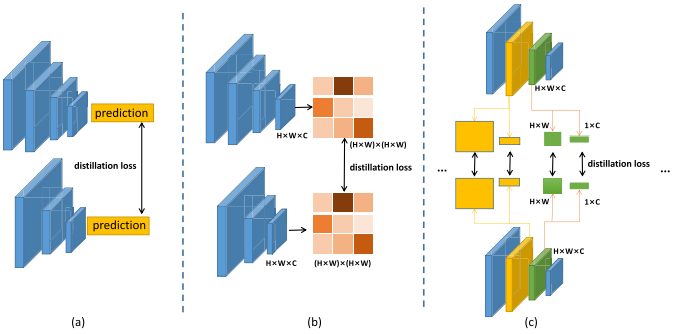


Fig. 3. Comparison between our distillation method with previous other methods. (a) Distillation method commonly used in image classification, and the distillation loss is only calculated for the final output. (b) Knowledge distillation method used in the segmentation network for compression and acceleration, and it is the distillation of the attention map. (c) Our DFD designed for incremental segmentation of aerial images. The features of each layer are compressed in spatial, and channel dimensions, and then the compressed features are, respectively, distilled so as to reduce the amount of calculation while maintaining the learning of features of various scales.

of the most rapidly developed and effective methods. We have used it in our network training.

C. Knowledge Distillation

Knowledge distillation aims to use one neural network as a teacher to train another student network. It is first proposed by [42] and has been applied in various fields [43]–[46]. For example, it is used for model compression, i.e., a small model with lower computational complexity can achieve the comparable performance of another large model [47]. Attention transfer [48] can transfer the attention information of the teacher network to the student network. Zagoruyko and Komodakis [48] calculate the gradient of the loss with respect to a specific input. A large gradient indicates that the loss function is highly sensitive to the input, so a larger weight is associated with it. DarkRank [49] proposes a knowledge distillation method for metric learning. The knowledge distillation in these works generally works on the output level, and a good training result can be obtained for conventional classification tasks. For semantic segmentation with dense classification, more supervised information is needed to guarantee the success of distillation training. Thus, we propose a multilayer module for evaluating the similarity of intermediate features between two models. The proposed method can maximize the capability of feature representations from the old model at different layers to ensure the inheritance of capabilities of old models.

III. Method

A. Overview

The overall training process of the class-incremental segmentation network is shown in Fig. 2, where we use Deeplabv3 [3] as the fundamental segmentation network. In the incremental learning, a new class C_t is learned at each step t, $t = 1, 2, \ldots, K$. At step t of training, pixels of ground-truth class t are added in training dataset with labels t, and then, some of the remaining pixels are labeled as classes $1, 2, \ldots, t-1$ based on the high confidence outputted by the newly learned classifier at step $t-1$, $t > 1$. Finally, all other pixels with no labels in an image are labeled as background. The pixels with labels $1, 2, \ldots, t$ and background form the training data of Deeplabv3 at step t. As shown in Fig. 2, the DFD module and the LM module as the contributions of our work provide the distillation computation at different levels. Different from previous works of preserving old training data, the DFD exploits the feature representations from all the layers of the encoders of segmentation models learned by the previous training step and learns to fit the feature representation mapping defined by old models via distillation. The LM module aims to combine the output labels $1, 2, \ldots, t-1$ of old models with the current label t to obtain comprehensive supervised information to enhance the training of the segmenter at step t. Three kinds of loss are defined in our network. Both modules and loss functions, as well as label alignment [18], will be presented in detail in the following.

B. Deep Feature Distillation Module

Knowledge distillation has achieved great success in image classification [45]. It aims to make one model (student) learn the essence of another model (teacher). It can be applied in the model compression [47], where a student model with a much smaller size has the comparative performance to a teacher model. Our work uses the distillation method in incremental segmentation to make a new model (student) have the comparative segmentation capability for old classes with old models (teacher). Specifically, a DFD module is presented to make model M_t have the comparative representation capability as model M_{t-1} at feature levels.

The difference in our distillation method from previous distillation methods is shown in Fig. 3. Concretely, our distillation method meets the distillation requirements of multilayer features and has low computational complexity. Fig. 3(a) describes a distillation method commonly used in the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 12:56:56 UTC from IEEE Xplore. Restrictions apply.

image classification task [36], which outputs one class for each image, and the distillation loss is calculated only on the final output. However, for the dense classification task as semantic segmentation, the convergence and performance of a student model can be guaranteed only depending on the distillation supervision of the output results.

Fig. 3(b) describes the distillation method used in semantic segmentation network compression [47], where a lightweight student network is trained to have a comparative performance to a teacher network. It calculates the similarity of the attention maps of the teacher and student network instead of the similarity of features. The choice is reasonable since the network architectures of the teacher and student model are different, but the distillation method is very computation-intensive due to evaluating large-size attention maps.

In our class-incremental learning, since the teacher network and the student networks share the same architecture, it is feasible to carry out the distillation at feature levels. To this end, we present a DFD module, as shown in Fig. 3(c). The operation of the DFD module mainly includes two steps: compression of spatial and channel dimensions and different levels of distillation.

1) Compression of Spatial and Channel Dimensions: Let $\mathbf{h}_\ell^t$ denote output feature map of layer l at step t of training and $\mathbf{h}_{\ell,c,w,h}^t$ denote the corresponding feature value at channel c and spatial position (w, h). A simple pixel-based distillation loss L_{pix} can be evaluated by

$$\mathcal{L}_{\text{pix}}\left(\mathbf{h}_\ell^{t-1}, \mathbf{h}_\ell^t\right) = \sum_{c=1}^{C_\ell}\sum_{w=1}^{W_\ell}\sum_{h=1}^{H_\ell}\left\|\mathbf{h}_{\ell,c,w,h}^{t-1} - \mathbf{h}_{\ell,c,w,h}^{t}\right\|^2 \quad (1)$$

where $\|\cdot\|$ denotes two-norm, and W_ℓ, H_ℓ, and C_ℓ denote the width, the height, and the number of channels of feature map $\mathbf{h}_\ell^t$, respectively. However, the distillation loss involves high computation complexity, and the corresponding loss value is so high to overwhelm other losses. At the same time, our experimental results show that it is too strict to be appropriate for optimizing incremental tasks. To reduce the computational complexity, we first compress feature maps of each layer l from spatial and channel dimensions to obtain a low-dimensional feature representation by

$$f_{\ell,c}^t = \sum_{w=1}^{W_\ell}\sum_{h=1}^{H_\ell}\mathbf{h}_{\ell,c,w,h}^t \quad (2)$$

$$g_{\ell,w,h}^t = \sum_{c=1}^{C_\ell}\mathbf{h}_{\ell,c,w,h}^t \quad (3)$$

where $f_{\ell,c}^t$ denotes the channel feature of feature map $\mathbf{h}_\ell^t$ and $g_{\ell,w,h}^t$ denotes the corresponding spatial feature. Furthermore, we can define the corresponding distillation loss as

$$\mathcal{L}_{\text{channel}}\left(\mathbf{h}_\ell^{t-1}, \mathbf{h}_\ell^t\right) = \sum_{c=1}^{C_\ell}\left\|f_{\ell,c}^{t-1} - f_{\ell,c}^t\right\|^2 \quad (4)$$

$$\mathcal{L}_{\text{space}}\left(\mathbf{h}_\ell^{t-1}, \mathbf{h}_\ell^t\right) = \sum_{w=1}^{W_\ell}\sum_{h=1}^{H_\ell}\left\|g_{\ell,w,h}^{t-1} - g_{\ell,w,h}^t\right\|^2. \quad (5)$$

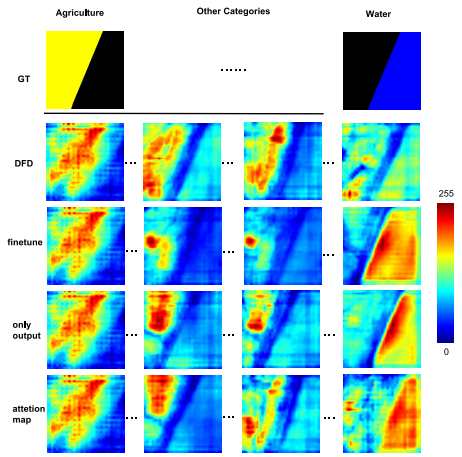


Fig. 4. Extracted features at different steps. The figure shows learning agriculture first, then learning other classes, and then learning water. Warm colors indicate high response, and cool colors indicate low response.

This new distillation loss has low computational complexity since a low-dimensional representation with dimensions $W_\ell \times H_\ell + C_\ell$ is obtained from original dimensions $W_\ell \times H_\ell \times C_\ell$. In addition, compared with the loss defined by (1), the new loss can be considered as a relaxation version since it does not require the strict matching between two corresponding feature maps. Our experimental results show this relatively loose matching is beneficial for incremental learning.

2) Different Levels of Distillation: Since our distillation computation has low complexity, it is applied on each residual module of the ResNet [50], which is used as the encoder by Deeplabv3, as shown in Fig. 3. Multilayer feature distillation is necessary for semantic segmentation since it requires each pixel to be classified, and matching feature representation at different levels is beneficial for incremental learning to inherit the capability of old models.

3) DFD in Training and Testing: Since the DFD module only involves summation operation, no trainable parameters need to be updated in the training process, which minimizes the impact on the training speed. Practically, inference speed is much more important than training speed. The DFD module only exists in the training process, so there is no difference for the models generated by the class-incremental learning and the conventional learning during the test phase.

4) Feature Visualization at Different Steps: Inspired by the feature representation in [51] and [52], we visualize the features extracted at each step in Fig. 4, using different incremental training methods, including the conventional finetune method, the distillation only at the output, distillation only based on attention maps, and the proposed DFD method. When first learning class "agriculture," the feature maps of all the methods can respond most to class "agriculture." In the follow-up learning of other classes, the responses to class "agriculture" decrease in varying degrees for different methods. When the "water" class is learned at last, the finetune method, distillation only at the output, and distillation only on attention maps make the response to "agriculture"

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 12:56:56 UTC from IEEE Xplore. Restrictions apply.

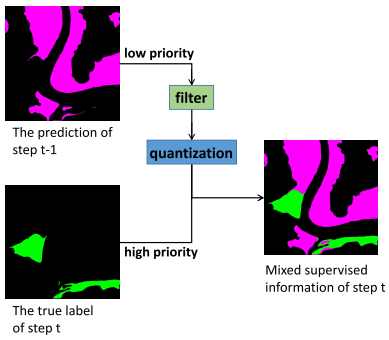


Fig. 5. Overview of the LM module. (Top) Output of the previous step. (Bottom) True label of the current step. LM combines the two parts as the comprehensive supervised information for the current step. The predictions of step $t-1$ need to go through filter, quantization, and then be combined, which will be introduced in detail.

decrease greatly and basically forget it, but the responses to class "water" constantly increases. Comparatively, our method (DFD) retains the memory of class "agriculture" and also responds well to the newly learned class "water."

C. Label Mixed Module

Before step t of training, the LM module tries to identify those pixels, which can be classified with high confidence by the model generated during step $t-1$ and then combine them with the pixels of class t plus their labels to form a training sample with comprehensive supervised information. It can be taken as the most direct way to strengthen the memory of old classes. Our experiments show that the stage most easily forgotten for a class is right after learning it, and the stage most accurately memorized is when it is being learned. For example, when class "river" is learned during step t, the generated model M_t has the highest recognition accuracy for it. However, after step $t+1$, the model M_{t+1} has the largest accuracy decrease for class "river," and this decline will gradually flatten out. Based on the two findings, the LM module retrain a class at the time when it is most easily forgotten during incremental learning, and the supervision information comes from the stage with the highest confidence, i.e., the LM module always provides the most accurate annotations at the most appropriate time to make the new model strengthen the memory of old classes.

The pipeline of the LM module is shown in Fig. 5. First, the output of the old model generated in step $t-1$ is filtered, and the pixels classified with high confidence are identified. Then, the output of the old model is quantified. Finally, the ground-truth labels of pixels of class t are combined with the quantified results to form the mixed labels for the supervised learning at step t.

1) Filtering via Confidence: We get a pixel classification model M_{t-1} after step $t-1$ of training. For each pixel x in an image, model M_{t-1} can output a probability distribution of t classes using the softmax function, where the tth class corresponds to the background. Let $P_i(x)$ denote the probability that pixel x belongs to class i, and we use the entropy $S(x)$ of classification distribution of pixel x to measure its confidence of pixel classification, i.e.,

$$S(x) = -\sum_i P_i(x) \log P_i(x). \tag{6}$$

Entropy represents the degree of chaos in physics. The higher entropy means the higher uncertainty in the decision. Thus, pixels with lower entropy correspond to the fact that its classification decision has higher confidence. To pick up the pixels with high classification confidence, a ratio threshold is needed. Concretely, for a batch of train images, all the pixels classified as classes 1 to $t-1$ by model M_{t-1} are collected (background pixels are excluded), and their entropy values are sorted in the increased order. Then, the first $(1/N)$ of pixels with low entropy values is identified, which will be given pseudolabels for step t of training. Here, N is a hyperparameter, and we will discuss the choice of its value in Section IV-F.

2) Quantization: For each identified pixel $\hat{x}$ with high confidence, a simple quantization operation is performed to give the pixel a one-hot label, i.e., set the class with the highest possibility to 1 and other classes to 0. The one-hot labels are consistent with the ground-truth labels, so they can be used as the supervised information for the next step t of training.

3) Combination Strategy: Before step t of training, for each pixel of an image, a label should be given to form the supervision information of step t of training. The label may come from one of two sources, i.e., the ground truth of class t and the output of high confidence from the old model M_{t-1}. It is possible that labels can be obtained from both sources, but they are different for the same pixel. When the case occurs, the ground truth of class t has higher priority since the confidence of ground-truth labels is apparently higher than pseudolabels' output by old models M_{t-1}. For the remaining pixels that do not get a label from both sources mentioned above, it is labeled as background.

The LM module can be applied in many scenarios where multiple classes are involved. If we want to apply class-incremental learning to classify multiple classes and make the LM module work well, the training data for step t should contain a large number of examples of the classes, which can be recognized well by the previous model M_{t-1}, though the examples are not given with labels. It is apparent that, in the semantic segmentation of aerial images, each image involves multiple classes, which makes a batch of training data for step t always consist of a mixture of new and old classes. The proposed LM module can achieve a good balance between learning new classes and avoiding forgetting old ones. Our experiments show that it can reduce the fluctuation of different weights of losses.

D. Loss

In step t class-incremental leaning, the total loss $\mathcal{L}^t$ is defined as a weighted sum of three kinds of losses, i.e.,

$$\mathcal{L}^t = \lambda_1 \mathcal{L}^t_{\text{cls}} + \lambda_2 \mathcal{L}^t_{\text{feature}} + \lambda_3 \mathcal{L}^t_{\text{output}} \tag{7}$$

where λ_1, λ_2, and λ_3 denote the weights of different kinds of losses, $\mathcal{L}^t_{\text{cls}}$ denotes the cross-entropy loss of classification

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 12:56:56 UTC from IEEE Xplore. Restrictions apply.

based on the supervised labels, $\mathcal{L}^t_{\text{feature}}$ denotes the distillation loss of features between old and current models, and $\mathcal{L}^t_{\text{output}}$ denotes the distillation loss of the new and old model output predictions. It should be noted that the total loss $\mathcal{L}^t$ only involves the classification loss $\mathcal{L}^t_{\text{cls}}$ for step 1.

The classification loss $\mathcal{L}^t_{\text{cls}}$ can be formulated as

$$\mathcal{L}^t_{\text{cls}} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{t+1}\hat{y}^t_{\text{ic}}\log\left(p^t_{\text{ic}}\right) \tag{8}$$

where N denotes the number of train examples (pixels), c denotes the index of classes, and $t+1$ classes are involved in the training of model M_t. $\hat{y}^t_{\text{ic}}$ denotes the cth entry of one-hot label vector for pixel i output by the LM module at step $t, t > 1$ (when $t = 1$, it is just the ground truth), and p^t_{ic} denotes the cth entry of the prediction output vector of model M_t for pixel i. The distillation loss of features $\mathcal{L}^t_{\text{output}}$ is evaluated by

$$\mathcal{L}^t_{\text{feature}} = \frac{1}{L}\sum_{\ell=1}^{L}\left(\mathcal{L}^t_{\text{space},l} + \mathcal{L}^t_{\text{channel},l}\right) \tag{9}$$

where L denotes the number of feature layers. $\mathcal{L}^t_{\text{space},l}$ and $\mathcal{L}^{t,l}_{\text{channel}}$ denote the distillation loss of spatial and channel features of the lth layer, respectively. The distillation loss of model output $\mathcal{L}^t_{\text{output}}$ is evaluated by

$$\mathcal{L}^t_{\text{output}} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{t} p^{t-1}_{\text{ic}}\log\hat{p}^t_{\text{ic}} \tag{10}$$

where p^{t-1}_{ic} denotes the cth entry of prediction output of model M_{t-1} for pixel i, and $\hat{p}^t_{\text{ic}}$ denotes the result obtained after the class prediction probability output by model M_t and is processed by the computation of label alignment. In fact, the outputs of model M_{t-1} and M_t correspond to different lengths of class distribution vectors (t and $t+1$ in our case), so we cannot use the cross-entropy to compute the distance of them. Generally, suppose that the prediction vector of the old model M_{t-1} is m-dimensional, and that of new model M_t is $m+n$-dimensional, where n is the number of new classes learned in step t; we accumulate the probabilities of the n classes new learned to the probability of background to obtain a new m-dimensional vector, i.e., we merge the newly learned n classes into class background. We call the procedure label alignment, as shown in Fig. 2. After label alignment, we can evaluate the output distillation loss based on cross-entropy metrics.

In the computation of losses, the output distillation loss and the classification loss seem to have some overlap since the high-confidence output of the old model is used to build comprehensive labels for evaluating the classification loss, and all the outputs of it are also used to evaluate the output distillation loss. In fact, both losses are necessary for training a good model M_t. First, to make M_t recognize a new class t, only classification loss can achieve this goal. At the same time, for the pixels that have high-confidence output from the old model M_{t-1}, the classification loss can also use them as training examples to directly train model M_t to obtain the classification capability of old classes. In addition, the high-confidence examples of old classes are generally a small fraction of all the training examples, so the output distillation is also necessary to exploit more training examples (except ones of class t) to indirectly obtain the classification capability of old classes from M_{t-1}. Thus, the two kinds of losses are complementary to each other. Our ablation studies in Section will verify their necessity.

IV. Experiments

In this section, extensive experiments are conducted to evaluate the effectiveness of the proposed method. In Section IV-A, the implementation details are introduced, including the setting of training parameters, datasets, evaluation metrics, and so on. In Section IV-B, we introduce some benchmark methods for performance comparison with our method. In Section IV-C, the results of comparison experiments are reported and analyzed. In Section IV-D, we use some example images to visually show the superiority of the proposed method. In Section IV-E, we conduct ablation studies to evaluate the respective significance of the DFD module, the LM module, and the designed loss function. In addition, different segmentation networks are evaluated to show the general application of our method. In Section IV-F, we discuss the influence of different thresholds for entropy on the network performance. In Section IV-G, using different training sequences, we evaluate the robustness of our work.

A. Implementation Details

For all the methods mentioned in this section, Deeplabv3 [3] is used as the base segmentation network, where ResNet-101 [50] is its backbone and initialized using the ImageNet pretraining model [53].

1) Training Parameters: We train the network just like [3], using SGD, the same learning rate strategy, momentum, and weight decay. The initial learning rate is 10^{-2} for the first learning step and 10^{-3} for the following steps. Since GPU memory requirements are an essential issue in semantic segmentation, in-place activation batch normalization is used, as proposed in [53]. The same data augmentation is applied as in [3]. During training and testing, the images are cropped to 512×512. For the hyperparameter setting of each method, the incremental learning protocol is employed, as defined in [54]. Our code implementation is completely based on the MiB project [18]. In the LM module, the hyperparameter is $N = 2$ for selecting high-confidence examples of old classes and backgrounds. Each step of class-incremental learning involves 50 epochs of training. Other settings of hyperparameters are the same as those in MiB. For example, weights of three losses are set as $\lambda_1 = 1$ and $\lambda_2 = \lambda_3 = 100$. The parameters of other networks for comparison are set as values given in the original papers. In subsequent experiments, the influence of different distillation loss weights on the results is evaluated.

2) Dataset: The dataset that we use is Deepglobe [55]. Each image of Deepglobe has 6M pixels. Deepglobe consists of 803 images, and each image has 2448×2448 resolutions. Just as GLNet [5], we partition these images into training

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 12:56:56 UTC from IEEE Xplore. Restrictions apply.

and testing sets with 455 and 348 images, respectively. The dense annotation contains six classes, i.e., urban, agriculture, rangeland, forest, water, and barren.

3) Preparation of Training Data and Testing Data: A large number of 512 × 512 patches are randomly cropped out from the original 2448 × 2448 images set. As a result, a training set containing 20 000 patches and a testing set containing 10 000 patches are set up, and we guarantee that both datasets can cover all six classes. We adopt the same way to prepare the training data for each step of class-incremental learning as MiB did. Concretely, for step t, if class t is the target class to be learned, all the patches that contain pixels of class t in the training set are identified and collected to form the training data for step t and for each patch; all other pixels except pixels of class t are labeled as background.

4) Evaluation Metric: We use mIOU as the evaluation metric. mIOU is the ratio of the intersection and union of two sets of true and predicted values, and can be evaluated by

$$\text{mIOU} = \frac{1}{K}\sum_{i=1}^{K}\frac{p_{ii}}{\sum_{j=1}^{K} p_{ij} + \sum_{j=1}^{K} p_{ji} - p_{ii}} \tag{11}$$

where K is the number of classes, i is the true class label, and j is the predicted label. p_{ij} denotes the number of pixels of class i predicted to class j.

B. Benchmark Methods for Comparison

The previous class-incremental learning is mainly aimed at the classification task. Since segmentation can be considered as a pixel-level classification task, we adjust the class-incremental learning methods in classification to address the segmentation issue. To demonstrate the superiority of the proposed method, we compare with some baseline methods, which corresponds to prior-focused methods and data-focused methods.

For prior-focused methods, we choose elastic weight consolidation (EWC) [33], path integral (PI) [35], and Riemannian walks (RWs) [34]. They employ different strategies to compute the importance of each parameter for old classes: EWC uses the empirical Fisher matrix, PI uses the learning trajectory, and RW combines EWC and PI in a unique model.

For data-focused methods, we choose learning without forgetting (LwF) [36], LwF multiclass (LwF-MC) [56], and incremental learning techniques (ILTs) [57]. LwF uses basic cross-entropy and distillation losses, and shares the same label space and classifier. LwF-MC is the single-head version of [36]. It is based on multiple binary classifiers, with the target labels defined using the ground truth for new classes and the probabilities given by the old model for the old ones. In line with MiB [18], we implement LwF-MC by a weighted combination of binary cross-entropy losses on both the ground truth and the probabilities given by the previous network. ILT [57] is the only method specifically proposed for semantic segmentation, and it uses a distillation loss in the output space.

C. Experimental Results

The comparison experiments are performed, and the experimental results are summarized in Tables I–III, where the best results among the benchmark methods are highlighted in blue. These tables list the learning order of classes, the IOU of each class obtained when the final model is evaluated on the testing set, and their mean mIOU. Since all the methods share the same network architecture, their model storage and computational complexity are basically the same. Tables I–III list the results of training one class, two classes, and three classes at each step. In the three cases, our method all obtains the best results. The fine-tune (FT) method forgets almost all the classes previously learned and only retains the classification ability of the last learned class. Although PI, EWC, and RW can keep the memory of old classes by regularization, their results are very poor in practice, which indicates that they are not suitable for semantic segmentation of aerial images. Comparatively, the data-focused methods, such as

TABLE I

RESULTS (%) OF TRAINING ONE CLASS AT A STEP

Method class	step 1 Urban	step 2 Agriculture	step 3 Rangeland	step 4 Forest	step 5 Water	step 6 Barren	overall mIOU
FT	0.0	0.0	0.0	0.0	0.0	86.7	14.4
PI [35]	0.1	0.1	0.4	1.1	0.3	71.7	12.4
EWC [33]	0.0	0.1	0.0	1.3	0.7	73.0	12.5
RW [34]	0.0	0.0	0.0	2.1	1.1	74.9	13.0
LWF [36]	40.2	44.4	2.0	29.4	45.7	28.5	31.7
LWF-MC [56]	32.9	56.8	11.5	39.3	72.2	44.4	42.8
ILT [57]	35.5	57.8	16.6	35.9	65.0	49.0	43.3
MiB [18]	52.1	50.4	20.2	20.4	34.7	33.9	35.3
ours	52.4	62.1	29.9	42.2	73.1	55.1	**52.4**

"FT" represents the fine-tuning approach.

TABLE II

RESULTS (%) OF TRAINING TWO CLASSES AT A STEP

Method class	step 1		step 2		step 3		overall mIOU
	Urban	Agriculture	Rangeland	Forest	Water	Barren	
FT	0.0	0.0	0.0	0	75.4	55.6	21.8
PI [35]	0.7	1.3	3.3	5.4	78.3	56.5	24.3
EWC [33]	0.6	1.1	2.0	5.2	77.5	54.3	23.5
RW [42]	0.7	1.2	4.2	6.9	74.6	52.5	23.4
LWF [36]	43.3	46.6	25.1	41.2	46.3	30.7	34.2
LWF-MC [56]	47.2	67.6	36.7	40.3	49.4	38.2	46.6
ILT [57]	53.2	58.9	27.7	40.6	50.0	43.4	45.6
MiB [18]	60.1	54.3	20.0	19.9	29.1	26.8	35.0
ours	55.6	66.7	46.4	47.9	57.8	45.3	**53.2**

TABLE III

RESULTS (%) OF THREE CLASSES FIRST AND THEN OTHER THREE CLASSES

Method class	step 1			step 2			overall mIOU
	Urban	Agriculture	Rangeland	Forest	Water	Barren	
FT	0.0	0.0	0.0	63.8	72.1	47.9	30.6
PI [35]	1.2	3.0	5.4	63.5	71.9	47.6	30.5
EWC [33]	1.1	3.3	4.7	57.3	75.5	44.3	32.1
RW [42]	1.3	3.2	5.2	60.9	72.7	46.5	31.0
LWF [36]	48.1	57.2	23.3	52.3	41.5	28.3	41.7
LWF-MC [56]	67.1	69.7	34.8	50.8	49.1	38.4	51.6
ILT [57]	62.2	63.7	23.9	53.6	46.2	35.2	47.4
MiB [18]	67.6	64.0	28.6	19.7	17.5	13.7	35.1
ours	70.1	69.1	43.0	55.3	50.7	44.4	**55.4**

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 12:56:56 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
RESULTS (%) ON POTSDAM AND VANHINGEN

Task	Potsdam		Vaihingen		Task	Potsdam		Vaihingen	
	[17]	Ours	[17]	Ours		[17]	Ours	[17]	Ours
4-1	76.7	77.6	75.7	74.9	3-2	75.9	77.2	74.8	74.6
2-3	75.1	76.7	74.4	74.4	2-2-1	-	76.3	73.9	74.1
joint (static)	75.3	77.8	74.7	75.1	-	-	-	-	-

LWF, LWF-MC, ILT, MiB, and our method, are obviously more effective than the prior-focus methods. Compared with LWF-MC and ILT, though MiB can better memorize old classes, its learning capability of new classes is much weaker. The experimental results show that our method ensures both the memory of old classes and the learning ability of new classes.

Compared with learning more classes within one step, the result of training one class at a time in Table I is the worst. It is understandable that the fewer classes are trained at one time, the more steps are needed, and the greater likelihood classes learned before are forgotten. However, this is basically the situation of the real world. Thus, we can see that our method is very effective and practical.

1) Results on Potsdam and Vaihingen: We conduct the comparison experiments of our method versus the SOTA method [17] on Potsdam and Vanhingen datasets, and the results are listed in Table IV, where *m-n* denotes that a network is trained with *m* classes at step 1 and then trained with *n* classes, and joint (called static learning in [17]) denotes the conventional learning way where all training examples are provided once. We can see that [17] is slightly better than ours in Vanhingen (4-1, 3-2) due to its enhancement of edge areas and small targets. However, our method is better than [17] when the number of tasks increases. In the real world, class-incremental learning generally involves more learning steps. In fact, the performance difference between [17] and ours is very slight on Potsdam and Vanhingen, and we feel it results from the characteristics of the datasets. Objects in Potsdam and Vaihingen are clustered, and one patch always involves multiple classes (even many old classes), which makes a network easily strengthen the memory of old classes via distillation loss when learning new classes. Comparatively, the Deepglobe [55] dataset seems to be more challenging, where the more the large objects are involved, the fewer the classes appear in a patch. Thus, it is difficult to obtain the source images of the previously learned classes during learning new classes. Deepglobe seems to better distinguish the effectiveness of different methods for incremental segmentation tasks.

D. Visual Results of Examples and Analysis

1) Results at Each Step: From the segmentation result of each step, as shown in Fig. 6, we can see that FT and RW methods completely forget old classes when they learn a new class, while ILT, LWF-MC, and MiB methods can achieve much better segmentation results due to more mild forgetting of old classes. For example, when learning the rangeland class in step 3, ILT, LWF-MC, and MiB methods misclassify some

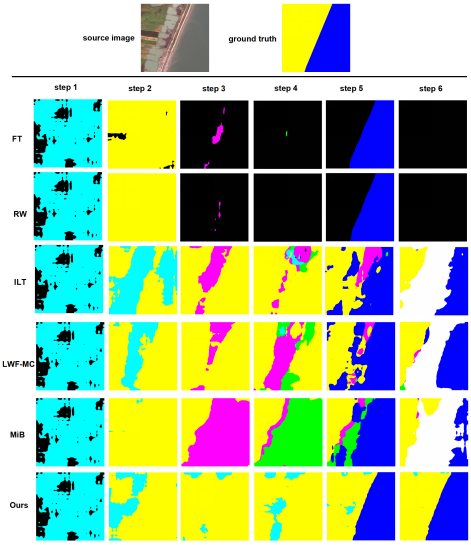


Fig. 6. Class-incremental segmentation results at each step. The order of learning is urban (cyan), agriculture (yellow), rangeland (purple), forest (green), water (blue), and barren (white).

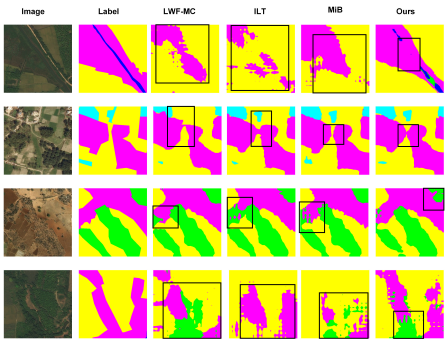


Fig. 7. Visual presentation of the results of different methods. Cyan for urban, yellow for agriculture, purple for rangeland, green for forest, blue for water, white for barren land, and black for the background. Black boxes indicate areas where predictions are wrong.

pixels of agriculture or water into class rangeland. Our method seems to have a good balance between learning new classes and memorizing old ones since the result is even approaching the ground truth.

2) Final Segmentation Results of Four Example Images: The final segmentation results of four example images are visually shown in Fig. 7. Intuitively, it can be shown that the segmentation results of our method are consistently much better than other methods, in terms of both the integrity of segmented objects and the accuracy of boundaries.

3) Error Analysis: As shown by the black boxes in Fig. 7, compared with ground truth, our method does not predict the forest (green) accurately, especially when it is adjacent to class rangeland (purple) and class agriculture (yellow). In addition, our method performs poorly for objects with slender shapes, such as rivers (water highlighted by blue). In class-incremental learning, each step usually can only face some of all the classes, so it can degrade the distinguishing ability of networks

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 12:56:56 UTC from IEEE Xplore. Restrictions apply.

TABLE V
RESULTS (%) OF ABLATION STUDY OF THE DFD MODULE

Method	mIOU	Method	mIOU
baseline	46.2	baseline+pixel-level dis.	45.1
baseline + DFD	52.4	baseline + attention map [48]	48.0

"pixel-level dis." represents the pixel-level distillation of features.

TABLE VI
RESULTS (%) OF ABLATION STUDY OF THE DFD MODULE WITH DIFFERENT LEVEL FEATURES

feature	mIOU	feature	mIOU	feature	mIOU
none	46.2	H	50.1	M	49.2
L	48.4	H+M	51.2	H+L	50.5
M+L	50.1	L+M+H	52.4	-	-

for similar classes compared with the networks obtained by one training using all the data.

E. Ablation Study

The DFD module and the LM module are two important components in the proposed class-incremental learning, so we need to evaluate their respective roles by ablation experiments. Since the setting of training one class at a time is closer to the real world and can better show the effect of a method, we use this setting in the following experiments.

1) No DFD Module: The experimental results are shown in Table V, and the baseline denotes the network where the DFD module is removed from the proposed method. It can be seen that, without the DFD module, the results decrease significantly. Attention map-based distillation has a certain effect, but the gap is still very large compared with the network with the DFD module. Unexpectedly, the results drop greatly when the pixel-level distillation loss is added to each pixel layer, which shows that there is a very large gap among the features extracted from aerial images for different classes. In this case, the very strict constraint is obviously very inappropriate. The network without the DFD module can only retain the memory of old classes by the classification loss derived by the LM module and the output distillation loss, so the performance degrades. In Table VII, the network without the LM module and the output distillation loss correspond to the FT method plus the DFD module, whose mIOU is only 30.1%. However, it is still 15.7% higher than the FT method (see Table I), which fully demonstrates the importance of the DFD module. Performance comparison results from Tables V and VII show that the DFD module plays a very important role in training class-incremental learning network for semantic segmentation.

2) Influence of the DFD Module With Different Level Features: In Deeplabv3, Resnet-101 is used as the encoder for feature extraction, which consists of 33 residual blocks divided into four layers. Our DFD is carried out on the outputs of the 33 residual blocks. The first and second layers correspond to low-level detail features (L), the third corresponds to middle-level structural features (M), and the fourth corresponds to high-level semantic features (H). To evaluate the effect of different levels of features on the final performance, the distillation experiments on different levels of features using the same settings are performed, and the results are shown in Table VI. It can be seen that middle-level features and high-level features have a greater impact on network performance than low-level features. Further analysis shows that the low-level features of the same class sometimes have great differences, while the differences of low-level features of different classes sometimes are relatively small. Thus, the low-level features have relatively weak discriminating ability compared to middle- and high-level features. Though high-level features (H) are the most significant features, low-level features (L) and middle-level features still provide a degree of representation of supplementary information since the results are improved by adding low- or middle-level features, which is also demonstrated in [58]. Thus, all three kinds of information distillation are necessary for the DFD module.

TABLE VII
RESULTS (%) OF ABLATION STUDY OF THE LM MODULE AND DISTILLATION LOSS

LM	output dis.	mIOU	LM	output dis	mIOU
✓	✓	52.4	✓	×	45.3
×	✓	37.4	×	×	30.1

"output dis." represents the output-level distillation.

3) No LM Module or Output Distillation Loss: In order to evaluate the impact of the LM module and the output distillation loss on network performance, we remove one of them or both to test the corresponding network performance, and the experimental results are summarized in Table VII. Without LM or output distillation loss, the performance will decrease a lot. Compared with the DFD module, the LM module obviously has a more significant impact on network performance. The mIOU of the learning system without output distillation loss is 45.3%, while that without the LM module is 37.4%, and the gap is obvious. The LM module identifies those examples with high-confidence output from the old model, and the classification loss is used to supervise the learning of the examples, which, indeed, can improve the network performance as shown by the experimental results. The LM module can be considered as a mask to select the examples with high confidence and filter ones with low confidence, so as to improve the final result. Most of the previous works only use the output distillation loss. The experimental results show that the introduction of the LM module can produce a large performance gain of 15%, and the combination with output distillation can effectively improve the segmentation accuracy.

4) Results on Different Segmentation Networks: All the previous experiments are conducted based on Deeplabv3. In order to evaluate the general applicability of our method, we conduct experiments on PSPNet [2] and DANet [59], respectively, to evaluate the effectiveness of the method. The experimental results are shown in Fig. 8. Compared with the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 12:56:56 UTC from IEEE Xplore. Restrictions apply.

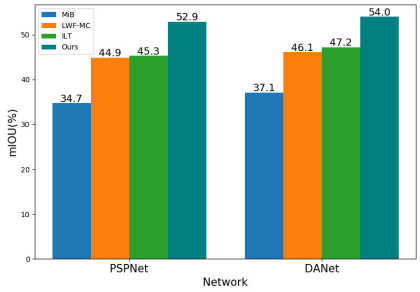


Fig. 8. Results (%) based on different segmentation networks.

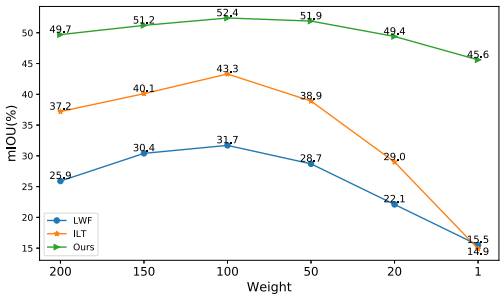


Fig. 9. Results (%) of LWF, ILT, and our method with different loss weights.

results generated by previous state-of-the-art methods [18], [56], [57], our method also achieves much better consistent results based on different segmentation networks, which demonstrates that the proposed method is very effective and has general applicability.

5) Sensitivity to Loss Weights: When a loss function involves different kinds of losses, the weight choice of different losses plays a significant role in the final result. Thus, we change the loss weights and test the network performance to investigate the influence of the weights on performance. In the experiments, following the usual practice of previous works [18], we make the weight of classification loss always be 1 and two weights of distillation the same. The experimental results are shown in Fig. 9. First, it can be seen that our method consistently outperforms the methods LWF and ILT in different weight configurations. In addition, compared with the previous works, LWF-MC and ILT, the sensitivity of our network to different weights of distillation loss is significantly reduced. We think that one of the possible explanations about the insensitivity is that the proposed LM module can dynamically adjust the direction of network optimization based on the proportion of new and old classes to well balance the final result. Thus, the experimental results show that the proposed method not only has a better performance but also it is relatively insensitive to the choice of loss weights.

F. Influence of the Threshold of Entropy

In the LM module, the output of the previous model $t-1$ needs to be used to compute the entropy of the class distribution of each pixel so that those high-confidence pixels of old classes can be selected, which is later used for evaluating classification loss. The ratio of selected pixels that involves the computation of classification loss is very important. A too-small ratio can cause the previously learned old classes to be forgotten; otherwise, some wrong class labels can be used as supervised information, which results in classification errors. Thus, we need to explore a suitable value for hyperparameter N through experiments. The experimental results are shown in Fig. 10. It can be seen that the results in the range of (1/3)–(1/2) are the good choices. The experimental results suggest that a ratio between them will bring relatively good performance.

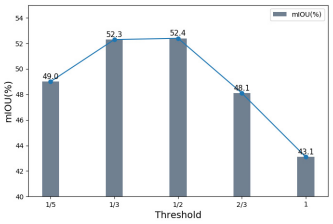


Fig. 10. Results (%) with different thresholds of the entropy.

TABLE VIII
RESULTS (%) OF DIFFERENT LEARNING ORDERS

Method	step 1	step 2	step 3	step 4	step 5	step 6	mIOU (%)	Average (%)
ILT	Agriculture	Barren	Water	Forest	Rangeland	Urban	45.1	42.6±2.2
	Urban	Agriculture	Rangeland	Barren	Water	Forest	43.3	
	Barren	Water	Forest	Rangeland	Agriculture	Urban	39.1	
	Forest	Urban	Barren	Agriculture	Water	Rangeland	42.7	
LWF-MC	Agriculture	Barren	Water	Forest	Rangeland	Urban	44.7	42.0±2.4
	Urban	Agriculture	Rangeland	Barren	Water	Forest	42.8	
	Barren	Water	Forest	Rangeland	Agriculture	Urban	38.2	
	Forest	Urban	Barren	Agriculture	Water	Rangeland	42.1	
ours	Agriculture	Barren	Water	Forest	Rangeland	Urban	55.9	**52.5**±2.7
	Urban	Agriculture	Rangeland	Barren	Water	Forest	52.4	
	Barren	Water	Forest	Rangeland	Agriculture	Urban	48.4	
	Forest	Urban	Barren	Agriculture	Water	Rangeland	53.1	

G. Influence of Different Learning Orders

For incremental learning, the order of learning classes has a significant influence on the performance of the final model. Specifically, the classes of high ratio in a dataset tend to get better results if they are learned in the early step since pixels of this class also appear in learning other classes later, i.e., they can be reviewed frequently. However, we usually have no knowledge about a prior class distribution in the real world. In order to evaluate the influence of different learning orders on the performance of the network, we conduct experiments on four different training sequences. The results are shown in Table VIII. It can be seen that different training sequences have a great impact on the final results. Aerial images are usually unbalanced in classes, which leads to that some classes may never be encountered again after the end of learning in the current stage, thus making the accuracy of this class very low. Our method outperforms the previous best methods, LWF-MC and ILT, for all training sequences, which fully demonstrates the robustness of our method.

V. CONCLUSION

This article presents an effective class-incremental learning method without storing old data for semantic segmentation.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 12:56:56 UTC from IEEE Xplore. Restrictions apply.

For alleviating the forgetting, our method achieves a good result due to the proposed two modules: the DFD module and the LM module. The DFD module can effectively learn good feature representations of old classes by introducing feature distilling loss. The proposed LM module can achieve a good balance between learning new classing and avoiding forgetting old ones by adding the examples of old classes with high confidence into the classification loss. The ablation studies show that both modules can bring great performance gains. By further integrating the existing output distillation, the results from the extensive experiments show that our method is significantly superior to some state-of-the-art methods on the dataset of aerial images.

REFERENCES

[1] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *Proc. Int. Conf. Med. Image Comput. Comput.-Assist. Intervent.* Berlin, Germany: Springer, 2015, pp. 234–241.

[2] H. Zhao, J. Shi, X. Qi, X. Wang, and J. Jia, "Pyramid scene parsing network," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 2881–2890.

[3] L.-C. Chen, G. Papandreou, F. Schroff, and H. Adam, "Rethinking atrous convolution for semantic image segmentation," 2017, *arXiv:1706.05587*.

[4] H. Zhao, X. Qi, X. Shen, J. Shi, and J. Jia, "ICNet for real-time semantic segmentation on high-resolution images," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 405–420.

[5] W. Chen, Z. Jiang, Z. Wang, K. Cui, and X. Qian, "Collaborative global-local networks for memory-efficient segmentation of ultra-high resolution images," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 8924–8933.

[6] L. Ding, J. Zhang, and L. Bruzzone, "Semantic segmentation of large-size VHR remote sensing images using a two-stage multiscale training architecture," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 8, pp. 5367–5376, Aug. 2020.

[7] L. Shan and W. Wang, "DenseNet-based land cover classification network with deep fusion," *IEEE Geosci. Remote Sens. Lett.*, early access, Jan. 12, 2021, doi: 10.1109/LGRS.2020.3042199.

[8] L. Shan *et al.*, "UHRSNet: A semantic segmentation network specifically for ultra-high-resolution images," in *Proc. 25th Int. Conf. Pattern Recognit. (ICPR)*, Jan. 2021, pp. 1460–1466.

[9] X. Yao, J. Han, G. Cheng, X. Qian, and L. Guo, "Semantic annotation of high-resolution satellite images via weakly supervised learning," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 6, pp. 3660–3671, Jun. 2016.

[10] X. Yao, X. Feng, J. Han, G. Cheng, and L. Guo, "Automatic weakly supervised object detection from high spatial resolution remote sensing images via dynamic curriculum learning," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 1, pp. 675–685, Jan. 2021.

[11] G. Cheng, C. Yang, X. Yao, L. Guo, and J. Han, "When deep learning meets metric learning: Remote sensing image scene classification via learning discriminative CNNs," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 5, pp. 2811–2821, May 2018.

[12] Z. Li *et al.*, "Cascaded multiscale structure with self-smoothing atrous convolution for semantic segmentation," *IEEE Trans. Geosci. Remote Sens.*, early access, Jun. 28, 2021, doi: 10.1109/TGRS.2021.3088902.

[13] Q. Ming, L. Miao, Z. Zhou, and Y. Dong, "CFC-Net: A critical feature capturing network for arbitrary-oriented object detection in remote-sensing images," *IEEE Trans. Geosci. Remote Sens.*, early access, Jul. 16, 2021, doi: 10.1109/TGRS.2021.3095186.

[14] X. Zheng, X. Chen, X. Lu, and B. Sun, "Unsupervised change detection by cross-resolution difference learning," *IEEE Trans. Geosci. Remote Sens.*, early access, May 31, 2021, doi: 10.1109/TGRS.2021.3079907.

[15] I. J. Goodfellow, M. Mirza, D. Xiao, A. Courville, and Y. Bengio, "An empirical investigation of catastrophic forgetting in gradient-based neural networks," 2013, *arXiv:1312.6211*.

[16] M. McCloskey and N. J. Cohen, "Catastrophic interference in connectionist networks: The sequential learning problem," in *Psychology of Learning and Motivation*, vol. 24. Amsterdam, The Netherlands: Elsevier, 1989, pp. 109–165.

[17] J. Li *et al.*, "Class-incremental learning network for small objects enhancing of semantic segmentation in aerial imagery," *IEEE Trans. Geosci. Remote Sens.*, early access, Nov. 1, 2021, doi: 10.1109/TGRS.2021.3124303.

[18] F. Cermelli, M. Mancini, S. R. Bulo, E. Ricci, and B. Caputo, "Modeling the background for incremental learning in semantic segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 9233–9242.

[19] O. Tasar, Y. Tarabalka, and P. Alliez, "Incremental learning for semantic segmentation of large-scale remote sensing data," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 12, no. 9, pp. 3524–3537, Sep. 2019.

[20] F. Ozdemir, P. Fuernstahl, and O. Goksel, "Learn the new, keep the old: Extending pretrained models with new anatomy and images," in *Proc. Int. Conf. Med. Image Comput. Comput.-Assist. Intervent.* Berlin, Germany: Springer, 2018, pp. 361–369.

[21] F. Ozdemir and O. Goksel, "Extending pretrained segmentation networks with additional anatomical structures," *Int. J. Comput. Assist. Radiol. Surg.*, vol. 14, no. 7, pp. 1187–1195, Jul. 2019.

[22] Y. Feng, X. Sun, W. Diao, J. Li, X. Gao, and K. Fu, "Continual learning with structured inheritance for semantic segmentation in aerial imagery," *IEEE Trans. Geosci. Remote Sens.*, early access, May 10, 2021, doi: 10.1109/TGRS.2021.3076664.

[23] U. Michieli and P. Zanuttigh, "Incremental learning techniques for semantic segmentation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. Workshop (ICCVW)*, Oct. 2019, pp. 3205–3212.

[24] M. Klingner, A. Bär, P. Donn, and T. Fingscheidt, "Class-incremental learning for semantic segmentation re-using neither old data nor old labels," in *Proc. IEEE 23rd Int. Conf. Intell. Transp. Syst. (ITSC)*, Sep. 2020, pp. 1–8.

[25] M. De Lange *et al.*, "A continual learning survey: Defying forgetting in classification tasks," 2019, *arXiv:1909.08383*.

[26] M. McCloskey and N. J. Cohen, "Catastrophic interference in connectionist networks: The sequential learning problem," in *Psychology of Learning and Motivation*, vol. 24. Amsterdam, The Netherlands: Elsevier, 1989, pp. 109–165.

[27] F. M. Castro, M. J. Marín-Jiménez, N. Guil, C. Schmid, and K. Alahari, "End-to-end incremental learning," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 233–248.

[28] H. Shin, J. K. Lee, J. Kim, and J. Kim, "Continual learning with deep generative replay," 2017, *arXiv:1705.08690*.

[29] S. Hou, X. Pan, C. C. Loy, Z. Wang, and D. Lin, "Learning a unified classifier incrementally via rebalancing," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 831–839.

[30] C. Wu *et al.*, "Memory replay GANs: Learning to generate new categories without forgetting," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 31, 2018, pp. 5962–5972.

[31] A. Mallya and S. Lazebnik, "PackNet: Adding multiple tasks to a single network by iterative pruning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 7765–7773.

[32] A. Mallya, D. Davis, and S. Lazebnik, "Piggyback: Adapting a single network to multiple tasks by learning to mask weights," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 67–82.

[33] K. James *et al.*, "Overcoming catastrophic forgetting in neural networks," *Proc. Nat. Acad. Sci. USA*, vol. 114, no. 13, pp. 3521–3526, Mar. 2017.

[34] A. Chaudhry, P. K. Dokania, T. Ajanthan, and P. H. Torr, "Riemannian walk for incremental learning: Understanding forgetting and intransigence," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 532–547.

[35] F. Zenke, B. Poole, and S. Ganguli, "Continual learning through synaptic intelligence," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 3987–3995.

[36] Z. Li and D. Hoiem, "Learning without forgetting," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 40, no. 12, pp. 2935–2947, Dec. 2018.

[37] P. Dhar, R. V. Singh, K.-C. Peng, Z. Wu, and R. Chellappa, "Learning without memorizing," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 5138–5146.

[38] Y. Wu *et al.*, "Large scale incremental learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 374–382.

[39] S. Dang, Z. Cao, Z. Cui, Y. Pi, and N. Liu, "Class boundary exemplar selection based incremental learning for automatic target recognition," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 8, pp. 5782–5792, Aug. 2020.

[40] S. Dang, Z. Cao, Z. Cui, Y. Pi, and N. Liu, "Open set incremental learning for automatic target recognition," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 7, pp. 4445–4456, Jul. 2019.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 12:56:56 UTC from IEEE Xplore. Restrictions apply.

[41] R. Aljundi, F. Babiloni, M. Elhoseiny, M. Rohrbach, and T. Tuytelaars, "Memory aware synapses: Learning what (not) to forget," in *Proc. Eur. Conf. Comput. Vis.*, 2018, pp. 139–154.
[42] P. Dhar, R. V. Singh, K.-C. Peng, Z. Wu, and R. Chellappa, "Learning without memorizing," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 5138–5146.
[43] X. Dai *et al.*, "General instance distillation for object detection," 2021, *arXiv:2103.02340*.
[44] M. Phuong and C. Lampert, "Distillation-based training for multi-exit architectures," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 1355–1364.
[45] F. Tung and G. Mori, "Similarity-preserving knowledge distillation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 1365–1374.
[46] H.-J. Ye, S. Lu, and D.-C. Zhan, "Distilling cross-task knowledge via relationship matching," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 12396–12405.
[47] Y. Liu, K. Chen, C. Liu, Z. Qin, Z. Luo, and J. Wang, "Structured knowledge distillation for semantic segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 2604–2613.
[48] S. Zagoruyko and N. Komodakis, "Paying more attention to attention: Improving the performance of convolutional neural networks via attention transfer," 2016, *arXiv:1612.03928*.
[49] Y. Chen, N. Wang, and Z. Zhang, "DarkRank: Accelerating deep metric learning via cross sample similarities transfer," in *Proc. AAAI Conf. Artif. Intell.*, 2018, vol. 32, no. 1, pp. 1–8.
[50] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.
[51] Q. He, X. Sun, Z. Yan, and K. Fu, "DABNet: Deformable contextual and boundary-weighted network for cloud detection in remote sensing images," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–16, 2022.
[52] X. Sun, P. Wang, C. Wang, Y. Liu, and K. Fu, "PBNet: Part-based convolutional neural network for complex composite object detection in remote sensing imagery," *ISPRS J. Photogramm. Remote Sens.*, vol. 173, pp. 50–65, Mar. 2021.
[53] S. R. Bulo, L. Porzi, and P. Kontschieder, "In-place activated BatchNorm for memory-optimized training of DNNs," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 5639–5647.
[54] M. De Lange *et al.*, "Continual learning: A comparative study on how to defy forgetting in classification tasks," 2019, vol. 2, no. 6, *arXiv:1909.08383*.
[55] I. Demir *et al.*, "DeepGlobe 2018: A challenge to parse the Earth through satellite images," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Jun. 2018, pp. 172–17209.
[56] S.-A. Rebuffi, A. Kolesnikov, G. Sperl, and C. H. Lampert, "ICaRL: Incremental classifier and representation learning," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 2001–2010.
[57] U. Michieli and P. Zanuttigh, "Incremental learning techniques for semantic segmentation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. Workshop (ICCVW)*, Oct. 2019, pp. 1–8.
[58] L. Zhu, D. Ji, S. Zhu, W. Gan, W. Wu, and J. Yan, "Learning statistical texture for semantic segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 12537–12546.
[59] J. Fu *et al.*, "Dual attention network for scene segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 3146–3154.



Lianlei Shan (Graduate Student Member, IEEE) received the B.E. degree from the Shandong University of Science and Technology, Qingdao, China, in 2018. He is currently pursuing the Ph.D. degree with the School of Computer and Control Engineering, University of Chinese Academy of Sciences, Beijing, China.

His research interests include remote sensing image processing and computer vision.



Weiqiang Wang (Member, IEEE) received the B.E. and M.E. degrees in computer science from Harbin Engineering University, Harbin, China, in 1995 and 1998, respectively, and the Ph.D. degree in computer science from the Institute of Computing Technology, Chinese Academy of Sciences (CAS), Beijing, China, in 2001.

He is currently a Professor with the School of Computer and Controlling Engineering, University of CAS, Beijing. His research interests include multimedia content analysis and computer vision.



Ke Lv (Member, IEEE) received the B.S. degree from the Department of Mathematics, Ningxia University, Yinchuan, China, in 1993, and the master's and Ph.D. degrees from the Department of Mathematics and the Department of Computer Science, Northwest University, Xi'an, China, in 1998 and 2003, respectively.

He is currently a Professor with the University of Chinese Academy of Sciences, Beijing, China. His research focuses on curve matching, 3-D image construction, and computer graphics.



Bin Luo (Senior Member, IEEE) was born in Anhui, China, in 1963. He received the B.Eng. degree in electronics and the M.Eng. degree in computer science from Anhui University, Hefei, China, in 1984 and 1991, respectively, and the Ph.D. degree in computer science from the University of York, York, U.K., in 2002.

From 1996 to 1997, he worked as a British Council Visiting Scholar at the University of York under the Sino-British Friendship Scholarship Scheme (SBFSS). He is currently a Full Professor with Anhui University.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 12:56:56 UTC from IEEE Xplore. Restrictions apply.